Internship Project Assignment

Project: 1

Title: Java Application Deployment with Reverse Proxy on AWS

Objective:

Deploy a **Java-based Student Registration Web Application** on AWS with proper database integration and secure reverse proxy access using open-source tools.

Requirements:

1. Infrastructure Setup:

- Launch **two Linux-based EC2 instances**:
  - One EC2 instance for hosting the **Java application backend**
  - One EC2 instance to act as a **reverse proxy**

2. Application Deployment:

- Host the .war-based Java web application using a **Servlet container** (e.g., Apache Tomcat).
- Ensure it is running and accessible **internally** on port 8080.
- Application file to be used:
  🔗 Download student.war

3. Database Setup:

- Launch an **Amazon RDS MySQL-compatible database**.
- Create:
  - A **new database schema**
  - A **student registration table** using proper SQL DDL syntax
- Ensure the Java application can **store form data** into the database.

4. Database Connector Configuration:

- Download and place the **MySQL Connector** in the servlet container's lib directory to enable DB connectivity.

🔗 Download mysql-connector.jar

5. Reverse Proxy Configuration:

- Install and configure a **reverse proxy** (e.g., Apache HTTPD or Nginx) on the second EC2 instance.
- Configure it to **forward traffic to the backend application server** (running on port 8080).
- Ensure only the proxy is **publicly accessible**.
- Block direct access to the backend EC2 using **Security Groups or firewalls**.

6. Access and URL Setup:

- The application must be accessible **only via the reverse proxy** at:
- http://<Proxy-Instance-Public-IP>/student
- Ensure direct access to the backend EC2 instance is **restricted**.

Deliverables:

1. **Working link** to the deployed application (via proxy)
2. **Screenshot of the database** showing stored registration data
3. **Summary report** (report.md or PDF) including:
    - Explanation of **traffic flow**: Proxy → Backend
    - List of **tools, services, and configurations** used
    - **Challenges encountered** and how you resolved them

Project: 2

Title: Set Up AWS Backup Plan for EC2 and RDS

Objective:

Learn how to configure and manage automated **backup and recovery** for AWS resources using **AWS Backup**. This project involves launching an EC2 and RDS instance, and creating a centralized backup plan to protect both.

Project Tasks:

1. Infrastructure Setup:

- **Launch one EC2 instance**:
  - Use Amazon Linux 2 or Ubuntu.
  - Install a sample web server (e.g., Apache or Nginx).
  - Store some test data on the instance (e.g., a sample HTML page or text file).
- **Launch one RDS instance**:
  - Choose MySQL or PostgreSQL engine.
  - Create a test database and insert sample data into a table.

2. AWS Backup Setup:

- Go to **AWS Backup** service in the console.
- Create a **Backup Vault** (or use the default vault).
- Create a **Backup Plan**:
  - Define backup rules (frequency, retention period, lifecycle).
  - Specify resources to be backed up using **Resource Assignments**.
- Assign both:
  - The EC2 instance
  - The RDS database

To the backup plan using resource tags or directly by resource ID.

3. Validation & Reporting:

- Trigger an **on-demand backup** to test the plan.
- Validate that both EC2 and RDS backups are created and listed under **Backup Jobs**.
- Take screenshots of:
  - Backup Plan
  - Backup Jobs

    - Recovery points for EC2 and RDS

Deliverables:

1. **Screenshots** of:
    - EC2 and RDS instances running
    - Backup Plan configuration
    - Backup Vault with recovery points
2. **Short Report** (1–2 pages) covering:
    - Steps taken to set up the backup
    - Key configuration details (frequency, retention, vault used)
    - Tools/console sections used
    - Any issues faced and how they were resolved

Project: 3

Title: Data Ingestion from S3 to RDS with Fallback to AWS Glue using Dockerized Python Application

Objective:

Develop a **Dockerized Python application** that automates the process of:

- Reading data from an **Amazon S3 bucket**
- Pushing it to an **RDS (MySQL-compatible) database**
- Automatically **falling back to AWS Glue** if the RDS database is unavailable or the push operation fails

This project helps you integrate multiple AWS services (S3, RDS, Glue), work with data pipelines, and use Docker for packaging and deployment.

Project Tasks:

1. Python Script Development:

- Create a Python script that:
  - Reads a CSV file from an S3 bucket
  - Parses it using **pandas**
  - Tries to upload the data to an **RDS MySQL-compatible database**
  - If the RDS upload fails, falls back to:
    - Creating a table in **AWS Glue Data Catalog**
    - Registering the dataset location in S3

2. Dockerfile Creation:

- Build a Dockerfile that:
  - Uses a Python 3.9+ base image
  - Installs all necessary libraries (boto3, pandas, sqlalchemy, pymysql)
  - Copies the Python script into the container
  - Runs the script on container startup

3. Requirements File:

- Create a requirements.txt with all necessary Python dependencies:

4. Image Build and Container Run:

- Build the Docker image:
- Run the container using environment variables for AWS credentials:

Configuration Parameters (Replace with actual values):

- **S3 Bucket Name**
- **CSV File Key**
- **RDS DB Endpoint, Username, Password, DB Name, Table Name**
- **Glue Database Name, Table Name, and S3 Location**

Deliverables:

1. **Python script** and **Dockerfile** stored in a GitHub repository
2. **Working Docker image and container logs** showing:
    - Successful push to RDS or fallback to Glue
3. **Screenshot** of:
    - Records inserted into RDS or
    - Table created in AWS Glue Catalog
4. **Summary Report** (1–2 pages) explaining:
    - Data flow (S3 → RDS → Glue fallback)
    - AWS services used
    - Docker setup
    - Challenges faced and how they were solved

Project: 4

Title: Deploy Prometheus and Grafana on Kubernetes Cluster using Terraform & Helm

Objective:

Set up **monitoring and visualization** tools — **Prometheus** and **Grafana** — on a Kubernetes cluster using **Infrastructure as Code (IaC)** tools: **Terraform** and **Helm**. This project strengthens skills in cloud provisioning, cluster monitoring, Helm charts, and IaC automation.

Project Tasks:

1. Kubernetes Cluster Provisioning:

- Use **Terraform** to provision a Kubernetes cluster on any cloud provider (EKS, AKS, GKE, or self-managed on EC2).
- Ensure kubectl and Terraform are configured to interact with the cluster.

💡 Optional: Use kind or minikube locally for practice if cloud isn't available.

2. Helm Installation Setup:

- Install **Helm** on your system.
- Initialize Helm in the cluster and add required chart repositories:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

3. Prometheus Deployment:

- Use Helm to deploy **Prometheus**:

```
helm install prometheus prometheus-community/prometheus --namespace monitoring --create-namespace
```

- Confirm the installation by accessing Prometheus UI via port-forward or LoadBalancer.

4. Grafana Deployment:

- Use Helm to deploy **Grafana**:

```
helm install grafana grafana/grafana --namespace monitoring
```

- Retrieve the Grafana admin password:

```
kubectl get secret --namespace monitoring grafana -o jsonpath="{.data.admin-password}" |
base64 --decode
```

- Access the Grafana dashboard and configure a Prometheus data source.

5. Terraform Integration:

- Use Terraform to:
  - Add Helm provider
  - Deploy Prometheus and Grafana as **Helm releases**
  - Manage namespace and dependencies declaratively

Deliverables:

1. Terraform code in GitHub repository.
2. Screenshots of:
   - Running Prometheus and Grafana pods (kubectl get pods -n monitoring)
   - Prometheus UI
   - Grafana dashboard
3. A short report explaining:
   - Infrastructure components used
   - Steps to deploy and connect the monitoring stack
   - Benefits of using Terraform and Helm together

Project: 5

Title: Deploying and Managing Microservices in a Cloud-Native Environment

Scenario:

You have recently joined **ABVC Solutions**, a fast-growing cloud-based software company, as a **DevOps Engineer**. The company has decided to **migrate its monolithic applications to a Kubernetes-based microservices architecture** to improve scalability, high availability, and deployment flexibility.

As part of the DevOps team, your responsibility is to **set up a Kubernetes environment**, **containerize microservices**, and implement all necessary Kubernetes components to ensure a production-like deployment scenario.

Objective:

Learn how to:

- Set up and configure a Kubernetes cluster
- Containerize and deploy a microservices application
- Manage Kubernetes deployments, services, autoscaling, and persistent storage

Assignment Tasks:

1. Kubernetes Setup

- Set up a Kubernetes cluster using one of the following tools:
  - Minikube for local development
  - Cloud-managed Kubernetes services like **EKS (AWS)**, **GKE (Google Cloud)**, or **AKS (Azure)**
- Ensure kubectl is installed and properly configured to interact with your cluster

2. Containerization and Docker

- Use or build a **sample microservices application** (e.g., user service, product service)
- Create separate **Dockerfiles** for each microservice
- Build Docker images and push them to a container registry (e.g., Docker Hub)

3. Kubernetes Deployments

- Write **Kubernetes Deployment YAML files** for each microservice
- Include configurations for:
    - Container image
    - Number of replicas
    - Port mappings
    - Environment variables (if needed)
- Deploy the microservices using `kubectl apply`

4. Service Discovery and Load Balancing

- Create Kubernetes **Service** resources to expose each microservice:
    - Use **ClusterIP** for internal communication
    - Use **NodePort** or **LoadBalancer** for external access (based on your cluster setup)
- Verify inter-service communication and external access

5. Scaling

- Configure **Horizontal Pod Autoscaler (HPA)** for one microservice (e.g., Product Service)
    - Set target CPU utilization threshold
- Generate artificial load (using tools like `hey`, `ab`, or `curl in loop`)
- Observe automatic scaling behavior via `kubectl get hpa`

6. Storage and Data Persistence

- Create a **Persistent Volume (PV)** and **Persistent Volume Claim (PVC)**
- Modify one microservice (e.g., Order Service) to read/write data to the persistent volume
- Demonstrate persistence by creating, reading, updating data via the service
    - Ensure data survives pod restarts

Deliverables:

1. Kubernetes YAML manifests:
    - Deployments
    - Services
    - HPA
    - PV/PVC
2. Dockerfiles and built image names
3. Screenshots of:
    - Cluster and pod status
    - HPA scaling
    - Persistent volume data being used

4. Short report (1–2 pages) detailing:
    - Cluster setup process
    - Deployment and testing steps
    - Tools and technologies used
    - Challenges faced and solutions implemented